

Case Study 11: API Testing for a Food Delivery Application

Project Name: FlashGrocer (Food Delivery Platform)

Real-World Scenario

Flash Grocer recently launched its updated food delivery application. Shortly after launch, customers began reporting critical functional defects:

1. Real-World Scenario Flash Grocer recently launched its updated food delivery application. Shortly after launch, customers began reporting critical functional defects:
2. Login Failures: Users are unable to authenticate properly.
3. Duplicate Orders: Users are being charged multiple times for the same food order due to lack of idempotency
4. Incorrect Payment Confirmations: Payment statuses are returning success even when transactions fail.

Problem Statement:

To design API testing using Postman to identify functional defects in the FlashGrocer backend APIs

Student Tasks

- Prepare API test cases.
- We identified three core API endpoints in the FlashGrocer backend that correspond to the reported defects
 - Authentication API: FOST /api/auth/login
 - Order Creation API: POST /api/orders/create
 - Payment API: POST/api/payments/confirm

Test Case ID | Module | Scenario to Test | Expected Status

TC-001 | Auth | Valid Login (Correct credentials) | 200 OK TC-002 | Auth | Invalid Login (Wrong password) | 401 Unauthorized TC-003 | Auth | Login Failure (Missing email) | 400 Bad Request TC-004 | Order | Create single valid order | 201 Created TC-005 | Order | Duplicate order submission | 409 Conflict TC-006 | Payment | Valid payment confirmation | 200 OK TC-007 | Payment | Invalid payment token | 400 Bad Request

- **Test authentication.**
 - Using Postman to hit the Flash Grocer authentication endpoint.
 - Test Case: TC-002 (Invalid Login)
 - Method: FOST
 - URL: <http://localhost:3000/api/auth/login>
 - Body (JSON)
 - email: "customer@flashgrocer.in",

- "password": "wrong password123",
- Postman Validation (Tests tab): `pm.test("Status code is 401", function () { 102 pm.response.status().should.equal(401);`
- `pm.test("Error message is present", function () { 111 var jsonData = pm.response.json();`
- `pm.expect(jsonData.success).to.equal(false);` Defect Identified: The system was returning 500 Internal Server Error instead of 401 Unauthorized on missing passwords.
- **Validate responses:**
- Testing the order creation API to ensure a user cannot submit the exact same food order twice in quick succession.
- Test Case: TC-005 (Duplicate Orders)
- Method: POST
- URL: <http://localhost:3000/api/orders/create>
- Headers: Idempotency-Key: req-abc-129
- Body (JSON):
- "items": {"productid": 3001,
- "quantity": 1,
- "total": 699
- Execution: Hit the Send button twice rapidly in Postman. Postman Validation;
- The first request should return 201 Created. The second request must return 409 Conflict or 200 OK (returning the original order, preventing a duplicate charge).
- Defect Identified: The backend was lacking an Idempotency-Key check, allowing two separate orders to be created simultaneously
- **Handle error scenarios.**
- Testing how the backend handles invalid payment tokens
- Test Case: TC-007 (Invalid Payment Token)
- Method: POST
- URL: <http://localhost:3000/api/payments/confirm>
- Body (JSON).
- "orderId": "ORE-99912",
- "paymentToken": "INVALID TOKEN XYZ"
- Postman Validation: `pm.test("Status code should be 400 Bad Request", function () { 122 um.response.status().should.equal(400);`

- Defect Identified: The system previously returned 200 OK with a successful status even when the payment gateway rejected the token. This functional defect. caused unpaid food orders to be dispatched

Expected Outcome

API testing report:

By leveraging Postman collections, variables, and automated test scripts, we successfully mapped out the functional defects in the Flash Grocer application. Summary of Fixes Required.

- 1.Authentication** : Implement proper 400/401 status code responses for invalid credentials instead of letting the app crash (500).
- 2.Orders**: Implement Idempotency-Key headers on the backend to reject duplicate requests for the same order cart within a short time frame.
- 3.Payments** : Add strict validation rules in the payment controller to ensure it verifies the token with the payment gateway before returning a success message to the frontend.

FlashGrocer API Testing – Case Study Infographic :

